MathScy: Domain-Specialized Mixture-of-Experts for AI-Assisted Mathematical Reasoning

Anonymous Authors
Anonymous Institution

Abstract

We present MathScy, a modular system for AI-assisted mathematical reasoning that integrates domain-specialized Mixture-of-Experts (MoE) large language models with formal verification in the Lean 4 proof assistant. MathScy addresses fundamental limitations of current AI systems for mathematics—including scalability across mathematical domains, depth of reasoning, and verifiability of outputs—through three key contributions. First, we construct a domain-specialized MoE architecture via Branch-Train-Mix, training eight LoRA expert modules on distinct mathematical subdomains (algebraic geometry, combinatorics, number theory, analysis, algebra, topology, probability, and computational mathematics) from a shared DeepSeek-Math-7B base model, with sigmoid gating and auxiliary-loss-free load balancing. Second, we design a Self-Play Theorem Prover (STP) loop in which a conjecturer and a prover iteratively improve through mutual training, with conjectures filtered through a four-tier Lean 4 verification pipeline encompassing type-checking, triviality detection, counterexample search, and automated proof search. Third, we curate a large-scale dataset of 6.47 million mathematical entries across 154 arXiv categories and 15 domain groups, from which we extract 241,586 structured training examples via a combination of LLM-powered knowledge extraction and LaTeX theorem mining. The system is designed to support conjecture formulation, proof construction, and counterexample finding, with a focus on applied, combinatorial, and computational algebraic geometry. We describe the complete architecture, training methodology, and data pipeline, and present preliminary results from the expert training phase, demonstrating stable convergence and efficient resource utilization across domain experts trained on a single A100 GPU. We outline the experimental protocol for evaluating MathScy on established benchmarks and open problems in algebraic geometry.

Keywords: Mixture-of-Experts, mathematical reasoning, theorem proving, conjecture generation, Lean 4, autoformalization, self-play, domain specialization

1 Introduction

Mathematical reasoning—encompassing conjecture formulation, proof construction, and counterexample identification—lies at the heart of scientific and technological progress. From the resolution of the Poincaré conjecture to the development of cryptographic protocols grounded in algebraic geometry, advances in mathematics have far-reaching consequences across the sciences [Hartshorne, 1977, Sturmfels, 2002]. Traditionally, the mathematical discovery process has been exclusively manual, requiring deep domain expertise and creative insight that is difficult to formalize or automate. Recent advances in artificial intelligence, particularly the Transformer architecture [Vaswani et al., 2017] and large language models (LLMs) built upon it—from GPT-2 [Radford et al., 2019] and BERT [Devlin et al., 2019] to GPT-3 [Brown et al., 2020] and instruction-tuned variants trained via reinforcement learning from human feedback [Ouyang et al., 2022]—have opened new possibilities for augmenting this process [Davies et al., 2021, Romera-Paredes et al., 2024, He, 2023], yet fundamental challenges remain.

Current AI systems for mathematical reasoning face four critical limitations. **Scalability**: mathematics encompasses a vast landscape of interconnected subdomains, each with its own vocabulary, proof techniques, and structural patterns; a model trained primarily on one area (e.g., combinatorics) may fail to transfer effectively to another (e.g., algebraic geometry) [Gururangan et al., 2020]. **Depth of reasoning**: while LLMs excel at pattern recognition and can generate plausible-looking mathematical text, they often struggle with the sustained logical reasoning required for complex proofs, particularly those requiring multiple intermediate lemmas or non-obvious proof strategies [OpenAI et al., 2024, Lewkowycz et al., 2022a]. **Verifiability**: LLM outputs are inherently probabilistic and prone to hallucination; without formal verification, there is no guarantee that a generated proof is correct or that a proposed conjecture is well-formed [Polu and Sutskever, 2020]. **Adaptability**: monolithic models require substantial retraining to incorporate new mathematical knowledge or adapt to emerging research areas [Ke et al., 2023].

We propose MathScy, a compositional and modular system designed to address these limitations through the integration of three key innovations:

1. **Domain-specialized Mixture-of-Experts architecture.** Rather than relying on a single general-purpose mathematical reasoner, MathScy employs a sparse MoE model with eight domain-specialized expert modules, each trained via Low-Rank Adaptation (LoRA) [Hu et al., 2022] on a distinct mathematical subdomain. The experts are constructed through the Branch-Train-Mix (BTM) paradigm [Sukhbaatar et al., 2024], which branches from a shared mathematical foundation (DeepSeek-Math-7B [Shao et al., 2024]), trains each branch on domain-specific data in embarrassingly parallel fashion, and mixes the specialized modules into a unified MoE architecture with sigmoid gating and auxiliary-loss-free load balancing [DeepSeek-AI et al., 2024b]. This design enables efficient scaling across mathematical domains while preserving within-domain depth.

2. **Self-Play Theorem Prover with formal verification.** Inspired by the expert iteration paradigm [Anthony et al., 2017, Silver et al., 2017] and recent advances in neural theorem proving [Polu et al., 2023, Ren et al., 2025], MathScy implements a self-play loop in which a conjecturer model and a prover model iteratively improve through mutual training. Generated conjectures are autoformalized into Lean 4 [de Moura and Ullrich, 2021] and subjected to a four-tier verification pipeline: type-checking, triviality detection via `aesop`, counterexample search, and automated proof attempts using Mathlib tactics [The mathlib Community, 2020]. Verification outcomes provide a noise-free training signal that grounds the self-play loop in formal correctness.

3. **Large-scale structured mathematical dataset.** We curate a corpus of 6.47 million mathematical entries spanning 154 arXiv categories and 15 domain groups, from which we extract 241,586 structured training examples through a combination of Gemini-powered knowledge extraction and LaTeX theorem mining from arXiv source files. The dataset is explicitly organized by mathematical type and domain, enabling stratified training of domain-specialized experts.

The system focuses on current research in applied, combinatorial, and computational algebraic geometry—areas where computational experimentation and AI-assisted discovery have particular potential to advance mathematical knowledge [Cox et al., 2015, Sommese and Wampler, 2005]. Algebraic geometry's rich interplay between algebraic structures and geometric intuition, combined with the computability of many fundamental invariants (Gröbner bases, Hilbert functions, intersection multiplicities), makes it an ideal testbed for AI-assisted mathematical reasoning.

This paper makes the following contributions:

- We present the first domain-specialized MoE architecture for mathematical reasoning, demonstrating that parameter-efficient expert modules trained on distinct mathematical subdomains can be effectively combined through learned routing.
- We design a complete self-play theorem proving pipeline that integrates conjecture generation, autoformalization, and formal verification in Lean 4, creating a closed loop between informal mathematical intuition and machine-checkable proofs.
- We construct one of the largest structured mathematical knowledge bases assembled for training domain-specialized models, with explicit type and domain annotations enabling stratified expert training.
- We provide a comprehensive experimental framework for evaluating AI-assisted mathematical reasoning systems, encompassing both established benchmarks (miniF2F, MATH, ProofNet) and targeted evaluations on open problems in algebraic geometry.

The remainder of this paper is organized as follows. Section 2 surveys related work across the relevant research areas. Section 3 establishes the mathematical foundations underlying our approach. Section 4 presents the MathScy system architecture and training methodology in detail. Section 5 describes the dataset construction pipeline. Section 6 outlines the experimental protocol. Section 7 presents preliminary results from the expert training phase. Section 8 discusses implications and limitations. Section 9 concludes.

2 Related Work

Our work lies at the intersection of several active research areas: mixture-of-experts architectures, large language models for mathematical reasoning, formal theorem proving, conjecture generation, mathematical dataset construction, domain specialization, and self-play for reasoning. We survey each area in turn, situating MathScy within the broader landscape.

2.1 Mixture-of-Experts Models

The Mixture-of-Experts (MoE) paradigm has its roots in the seminal work of Jacobs et al. [1991], who proposed a system of expert networks governed by a gating mechanism that partitions the input space, enabling different experts to specialize on different subsets of the data. This foundational idea—that a collection of specialized sub-models can outperform a single monolithic model—has undergone a remarkable renaissance in the era of large-scale Transformers. The key insight driving modern MoE research is that sparse conditional computation allows models to scale parameter counts dramatically while keeping per-token computational cost tractable, a property first demonstrated at scale by the Sparsely-Gated Mixture-of-Experts layer of Shazeer et al. [2017], which interposed a trainable gating network between Transformer layers and a large pool of feed-forward experts, activating only the top-k experts per token.

The subsequent evolution of MoE architectures has been defined by innovations in routing, load balancing, and scaling efficiency. Lepikhin et al. [2021] introduced GShard, which scaled MoE to 600 billion parameters for multilingual machine translation with a top-2 gating strategy and random auxiliary routing to a second expert, demonstrating the viability of MoE for cross-lingual generalization. The Switch Transformer [Fedus et al., 2022] simplified this design by employing top-1 routing—dispatching each token to a single expert—and introducing differentiable load-balancing auxiliary losses alongside capacity factors to prevent expert collapse, enabling models to scale to over one trillion parameters with reduced communication overhead. Concurrently, GLaM [Du et al., 2022] demonstrated that a 1.2-trillion-parameter MoE model with 64 experts per layer could match GPT-3 quality while using only one-third of the energy for training, and Riquelme et al. [2021] showed that MoE principles extend effectively to vision

transformers. Zhou et al. [2022] provided a comprehensive survey of MoE methods, systematically categorizing advances in gating mechanisms, training strategies, and applications. ST-MoE [Zoph et al., 2022] further refined the design space by introducing stable training techniques and encoder-decoder MoE models that achieved state-of-the-art performance on reasoning benchmarks. These works established auxiliary losses as the standard mechanism for encouraging balanced expert utilization, though at the cost of introducing a hyperparameter whose magnitude must be carefully tuned: too large an auxiliary loss degrades task performance, while too small a loss permits routing degeneration.

More recently, a new generation of MoE models has pushed beyond auxiliary-loss-based load balancing. The DeepSeekMoE architecture [Dai et al., 2024] proposed fine-grained expert segmentation combined with shared experts, isolating common knowledge in a subset of always-active experts while routing specialized tokens to domain-specific ones. DeepSeek-V2 [DeepSeek-AI et al., 2024a] extended this with Multi-head Latent Attention and further refined the expert-routing paradigm. Most relevant to our work, DeepSeek-V3 [DeepSeek-AI et al., 2024b] introduced an auxiliary-loss-free load balancing strategy that replaces explicit auxiliary losses with a dynamic bias term added to affinity scores during top-K routing: expert load is monitored at each training step, and the bias is adjusted to penalize overloaded experts and promote underloaded ones. Critically, DeepSeek-V3 also adopted sigmoid gating—computing affinity scores via the sigmoid function and normalizing among selected experts—rather than the conventional softmax over all experts, yielding more flexible multi-expert activation patterns. This combination of sigmoid gating with loss-free balancing achieves superior performance to auxiliary-loss methods, and directly informs the gating design in MathScy.

The open-source MoE ecosystem has also matured substantially. Mixtral 8×7B [Jiang et al., 2024] demonstrated that a sparse mixture of eight experts per layer, with top-2 routing and 12.9B active parameters out of 46.7B total, can match or exceed models several times its effective size, including Llama 2 70B. OLMoE [Muennighoff et al., 2024] provided a fully open, reproducible MoE baseline with 7B total parameters and 1B active, trained on 5 trillion tokens, accompanied by open-source weights, training data, and code—establishing important baselines for MoE research and demonstrating that even modestly-sized MoE models can achieve competitive performance through careful data curation and training.

Of particular importance to our architectural design is the Branch-Train-Mix (BTX) framework of Sukhbaatar et al. [2024], which proposes a three-stage recipe for constructing domain-specialized MoE models: (i) branching from a seed pretrained model by copying it, (ii) training each copy independently on a domain-specific data subset in embarrassingly parallel fashion, and (iii) mixing the specialized experts into MoE layers with averaged non-expert parameters, followed by MoE fine-tuning to learn token-level routing. BTX generalizes both Branch-Train-Merge [Li et al., 2022] (which lacks learned routing) and sparse upcycling (which omits independent expert training), demonstrating superior accuracy and efficiency on mathematics and code domains without catastrophic forgetting. MathScy adopts the BTX paradigm but replaces full-model branching with parameter-efficient LoRA expert modules, enabling us to train eight domain-specialized mathematical experts from a shared DeepSeek-Math base with substantially reduced memory and compute requirements.

2.2 Large Language Models for Mathematical Reasoning

The application of large language models to mathematical reasoning has progressed rapidly from early demonstrations of arithmetic and word-problem solving to systems approaching competition-level mathematics. Early benchmarks such as the DeepMind Mathematics Dataset [Saxton et al., 2019] and the ARC challenge [Clark et al., 2018] revealed both the promise and limitations of neural approaches to mathematical and abstract reasoning tasks. The scaling paradigm established by T5 [Raffel et al., 2020], PaLM [Chowdhery et al., 2023], Gopher [Rae et al., 2022], and the Chinchilla scaling laws [Hoffmann et al., 2022] demonstrated that model

and data scale are critical determinants of mathematical capability. The LLaMA family [Touvron et al., 2023a,b] democratized access to capable base models, while Galactica [Taylor et al., 2022] showed that pretraining on scientific corpora yields strong performance on mathematical tasks. A pivotal advance was the discovery by Wei et al. [2022] that chain-of-thought (CoT) prompting—providing a few exemplar reasoning chains as demonstrations—elicits multi-step reasoning capabilities that emerge with sufficient model scale (approximately 100B parameters), dramatically improving performance on arithmetic, commonsense, and symbolic reasoning benchmarks. This emergent property of large models underscored the importance of scale and prompted the development of mathematics-specialized architectures.

Minerva [Lewkowycz et al., 2022a], based on PaLM, demonstrated that continued pretraining on 118GB of scientific papers from the arXiv, combined with few-shot CoT prompting and majority voting, could achieve state-of-the-art performance on quantitative reasoning tasks spanning mathematics, physics, and other STEM domains. Follow-up work by Lewkowycz et al. [2022b] extended Minerva's approach to broader scientific reasoning tasks beyond mathematics. However, Minerva's answers could not be automatically verified, as the model operates entirely in natural language without recourse to formal grounding. GPT-4 [OpenAI et al., 2024] further raised the ceiling on mathematical reasoning, achieving strong performance on competition-level benchmarks including the AMC and AIME, though its closed-source nature limits its utility as a research platform.

The open-source mathematical reasoning community has produced a series of increasingly capable models. Llemma [Azerbayev et al., 2024], continuing pretraining Code Llama on the Proof-Pile-2 mixture of scientific papers, mathematical web text, and formal proof data, outperformed all open base models on the MATH benchmark and demonstrated capability in both tool use and formal theorem proving without task-specific fine-tuning. DeepSeekMath [Shao et al., 2024] pushed the boundaries further by leveraging a meticulously engineered web data selection pipeline to harvest high-quality mathematical content from Common Crawl, and introduced Group Relative Policy Optimization (GRPO)—a memory-efficient variant of PPO—to enhance mathematical reasoning through reinforcement learning, achieving 51.7% on the MATH benchmark at the 7B scale. InternLM-Math [Ying et al., 2024b] contributed a chain-of-thought reasoning framework integrated with tool use and formal verification capabilities, while Qwen2.5-Math [Yang et al., 2024] established the current state of the art among open models by supporting both CoT and Tool-Integrated Reasoning (TIR) across multiple languages.

The integration of external computational tools with natural language reasoning has emerged as a powerful paradigm for mathematical problem solving. ToRA [Gou et al., 2024] introduced Tool-integrated Reasoning Agents that interleave natural language reasoning steps with calls to computation libraries and symbolic solvers, trained via imitation learning on GPT-4-generated interactive trajectories. ToRA demonstrated that even modest-scale models (34B) can outperform GPT-4's CoT performance on competition-level benchmarks when equipped with appropriate tool interfaces. Process-level verification [Lightman et al., 2024] further enhanced reasoning reliability by training reward models to verify individual reasoning steps rather than only final answers, enabling more fine-grained feedback during both training and inference. Math-Shepherd [Wang et al., 2024b] extended process supervision with automatic step-level reward labeling, enabling scalable reward model training. More recently, DeepSeek-R1 [DeepSeek-AI et al., 2025] demonstrated that reinforcement learning alone can induce sophisticated mathematical reasoning behaviors, including self-verification and chain-of-thought, without supervised fine-tuning. AlphaGeometry [Trinh et al., 2024] combined a neural language model with a symbolic deduction engine to solve Olympiad-level geometry problems, approaching human gold-medalist performance. The MUSTARD benchmark [Huang et al., 2024] was introduced to evaluate multi-step textual reasoning with distractors, providing a more rigorous test of mathematical reasoning robustness.

MathScy builds upon these advances by combining domain-specialized MoE architecture

with mathematical reasoning, hypothesizing that different areas of mathematics—from algebraic geometry to combinatorics to probability theory—benefit from dedicated expert modules rather than a single general-purpose mathematical reasoner. Our approach complements the reasoning enhancements of CoT, GRPO, and tool integration by introducing an orthogonal axis of improvement: structural specialization at the architecture level.

2.3 Formal Theorem Proving with Large Language Models

The intersection of large language models and formal theorem proving represents one of the most promising avenues for achieving verifiable mathematical reasoning. Earlier work on premise selection for theorem proving by Ferreira and Freitas [2020] demonstrated that natural language processing techniques could identify relevant premises from large libraries, and TacticToe [Gauthier et al., 2021] applied Monte Carlo tree search to tactic-based proof search in HOL4. The pioneering GPT-f system [Polu and Sutskever, 2020] demonstrated that Transformer-based language models could generate novel proofs in the Metamath formal system, contributing 23 shortened proofs that were accepted into the main Metamath library—the first instance of a deep learning system contributing proofs adopted by a formal mathematics community. Subsequent work by Polu et al. [2023] on formal mathematics statement curriculum learning showed that expert iteration with a curriculum of progressively harder statements could substantially improve proof search, establishing a paradigm in which language models are iteratively trained on their own successful proofs.

The development of interactive proof assistants, particularly Lean 4 and its mathematical library Mathlib, has created a rich ecosystem for neural theorem proving. Early efforts in Lean-based autoformalization by Jiang et al. [2022] demonstrated the feasibility of translating informal mathematical statements into Lean, while Wu et al. [2022] showed that large language models can autoformalize mathematical statements at scale with surprisingly high accuracy. Zheng et al. [2024] explored verification techniques within the Lean ecosystem, and recent progress surveyed by Ying et al. [2024a] highlights the rapid maturation of Lean-based proving tools. LeanDojo [Yang et al., 2023b] provided a comprehensive toolkit for extracting training data from Lean repositories and interacting with the Lean prover programmatically, enabling the training of ReProver—a retrieval-augmented theorem prover that selects relevant premises from the library to guide proof search. LeanCopilot [Song et al., 2024] brought large language models directly into the Lean development environment as a copilot, suggesting tactics and premises in real time. LeanAnime [Yang et al., 2023a] introduced animated visualization techniques to make formal proofs more accessible. The LEGO-Prover [Wang et al., 2024a] introduced a modular approach to proof construction, maintaining a growing library of verified lemmas that can be retrieved and composed to prove new theorems, advancing the state of the art on the miniF2F benchmark [Zheng et al., 2022].

The most dramatic recent advance in formal theorem proving is AlphaProof [Hubert et al., 2025], which coupled a pretrained language model with the AlphaZero reinforcement learning algorithm to achieve silver-medal performance at the 2024 International Mathematical Olympiad, solving four out of six problems. AlphaProof guarantees correctness by verifying every logical step using the Lean proof assistant, and employs test-time reinforcement learning to enable deep, problem-specific adaptation. The DeepSeek-Prover series has also made significant contributions: DeepSeek-Prover [Xin et al., 2024a] introduced Monte Carlo tree search for proof generation in Lean 4, DeepSeek-Prover-V1.5 [Xin et al., 2024b] refined this approach with improved training data curation and proof search strategies, and DeepSeek-Prover-V2 [Ren et al., 2025] advanced formal reasoning through subgoal decomposition, achieving 88.9% on miniF2F-test with the 671B variant. Draft, Sketch, and Prove [Jiang et al., 2023] and Baldur [First et al., 2023] explored complementary strategies of proof sketching and whole-proof generation, respectively. NaturalProver [Welleck et al., 2022] generated proofs by conditioning on natural language references and definitions, bridging the gap between informal and formal reasoning. Theorem-

Llama [Chen et al., 2024] fine-tuned the LLaMA architecture specifically for theorem proving, demonstrating that open-weight models can be adapted into capable provers. MagnusHammer [Mikuła et al., 2024] advanced premise selection through transformer-based retrieval, significantly improving tactic-based proof search by identifying the most relevant library lemmas.

MathScy integrates formal verification as a core component of its Self-Play Theorem Prover loop, using Lean 4 autoformalization to translate naturally-expressed conjectures into formally verifiable statements. Unlike systems that focus exclusively on formal proof search, our approach treats autoformalization as a bridge between the conjecture generation capabilities of domain-specialized experts and the verification guarantees of formal proof assistants.

2.4 Conjecture Generation and Mathematical Discovery

The automation of conjecture generation—producing novel, non-trivial mathematical hypotheses—represents a fundamentally different challenge from theorem proving, requiring creative exploration of mathematical structure rather than verification of known claims. A watershed moment for AI-driven mathematical discovery came with Davies et al. [2021], who demonstrated that machine learning could guide human mathematical intuition by identifying previously unknown patterns in knot theory and representation theory. By training neural networks to predict relationships between mathematical invariants and then interpreting the learned representations, the collaboration led to the discovery of a new geometric quantity and new connections between Bruhat intervals and Kazhdan–Lusztig polynomials.

Fawzi et al. [2022] introduced AlphaTensor, which cast the discovery of efficient matrix multiplication algorithms as a reinforcement learning problem, discovering algorithms that surpassed Strassen's 50-year-old method for certain matrix sizes. FunSearch [Romera-Paredes et al., 2024], published in Nature, represented the first instance of an LLM-based system making genuine new mathematical discoveries, discovering new constructions of large cap sets exceeding previously known bounds by searching the space of programs rather than solutions. Poesia et al. [2024] proposed learning formal mathematics from intrinsic motivation, jointly training a conjecturer and prover in a self-play framework that bootstraps from axioms alone. Wagner [Wagner, 2021] approached conjecture generation through reinforcement learning for counterexample search, and He [2023] surveyed the emerging applications of machine learning to pure mathematics.

MathScy contributes to this landscape by coupling domain-specialized expert models with a self-play loop that jointly trains a conjecturer and a prover, with domain specialization enabling conjectures grounded in specific mathematical vocabularies and structural patterns.

2.5 Dataset Construction for Mathematical AI

The quality and coverage of training data have emerged as decisive factors in the performance of mathematical reasoning systems. The MATH dataset [Hendrycks et al., 2021] established a foundational benchmark of 12,500 competition-level problems with detailed solutions. OpenWebMath [Paster et al., 2024] extracted 14.7 billion tokens of mathematical web content from Common Crawl, while MathPile [Wang et al., 2023b] assembled a diverse 9.5-billion-token corpus from textbooks, arXiv papers, and mathematical websites. For formal mathematics, ProofNet [Azerbayev et al., 2023] provided undergraduate-level formal proving benchmarks in Lean 4, NaturalProofs [Welleck et al., 2021] assembled 30,000 theorem-proof pairs from mathematical repositories, and the Goedel dataset [Lin et al., 2025] produced 1.64 million formalized statements through synthetic data generation.

Beyond mathematics-specific datasets, broader evaluation frameworks such as HumanEval [Chen et al., 2021] for code generation, BigCodeBench [Zhuo et al., 2024] for compositional coding tasks, and HELM [Liang et al., 2023] for holistic language model evaluation have established methodological standards for rigorous model assessment that inform our evaluation design.

Our dataset of 6.47 million mathematical entries across 15 domains, explicitly structured around mathematical type and domain, represents one of the largest structured collections assembled for training domain-specialized models, complementing the pretraining-focused corpora (OpenWebMath, MathPile) and formal verification datasets (ProofNet, Goedel).

2.6 Domain Specialization and Parameter-Efficient Fine-Tuning

The effectiveness of domain-adaptive pretraining was established by Gururangan et al. [2020], who showed that a second phase of pretraining on in-domain text yields consistent performance gains across diverse domains. Parameter-efficient fine-tuning (PEFT) methods make multi-domain specialization tractable: prompt tuning [Lester et al., 2021] demonstrated that learning soft prompts can adapt frozen models to downstream tasks with minimal parameters, LoRA [Hu et al., 2022] represents weight updates as low-rank decompositions, QLoRA [Dettmers et al., 2023] enables fine-tuning through 4-bit quantized base models, ReFT [Wu et al., 2024] introduced representation fine-tuning that intervenes on hidden representations rather than weights, and DoRA [Liu et al., 2024] refines this by decomposing weight matrices into magnitude and direction components. The adapter framework of Houlsby et al. [2019] pioneered task-specific bottleneck modules with minimal parameter overhead. Scialom et al. [2022] provided a comprehensive survey of fine-tuning strategies for language models, while Gupta et al. [2023] addressed the challenge of continual learning in the context of domain adaptation, proposing methods to prevent catastrophic forgetting when models are sequentially adapted to new domains.

In MathScy, each expert consists of LoRA adapters specialized to a mathematical domain, while the shared base parameters of DeepSeek-Math-7B encode general mathematical knowledge—enabling eight domain experts to be trained with a fraction of the memory required by full-model branching.

2.7 Self-Play and Reinforcement Learning for Reasoning

Self-play reinforcement learning, as demonstrated by AlphaZero [Silver et al., 2017], achieves superhuman performance through agents learning by competing against themselves. Expert Iteration [Anthony et al., 2017] formalized this as a dual-process framework, and Proximal Policy Optimization [Schulman et al., 2017] provided a stable policy gradient algorithm that has become the standard for reinforcement learning from human feedback in language models. Self-Instruct [Wang et al., 2023a] demonstrated that language models can generate their own instruction-following training data, enabling self-improvement without human annotation. Kadavath et al. [2022] investigated language models' capacity for self-knowledge and calibrated self-evaluation, showing that models can assess the correctness of their own outputs—a capability central to self-play frameworks. STaR [Zelikman et al., 2022] introduced self-taught reasoning for language models, and ReST [Gulcehre et al., 2023] generalized this through an expectation-maximization framework. BIG-bench [Srivastava et al., 2023] provided a comprehensive benchmark suite for evaluating diverse reasoning capabilities, including mathematical reasoning tasks that inform self-play evaluation. Singh et al. [2024] demonstrated that iterative self-training with correctness feedback surpasses human-data fine-tuning for mathematical reasoning. In formal mathematics, Polu et al. [2023] showed that expert iteration with curriculum learning dramatically improves proof search.

MathScy extends the self-play paradigm by introducing domain-specialized experts into the conjecturer-prover loop, ensuring that conjectures remain mathematically meaningful within their respective fields.

3 Background and Preliminaries

We establish the mathematical foundations underlying MathScy's architecture. Our system integrates four key technical pillars: sparse Mixture-of-Experts models for domain-specialized reasoning, parameter-efficient fine-tuning via low-rank adaptation, formal verification through dependent type theory, and self-play theorem proving for iterative improvement.

3.1 Mixture-of-Experts Models

The Mixture-of-Experts (MoE) paradigm [Jacobs et al., 1991] enables conditional computation by routing each input to a subset of specialized sub-networks, thereby decoupling model capacity from per-example computational cost.

3.1.1 Standard MoE Formulation

Consider an input representation $\mathbf{x} \in \mathbb{R}^d$ at a given layer of a transformer network. An MoE layer replaces the monolithic feed-forward network (FFN) with N expert networks $\{E_i\}_{i=1}^N$, each $E_i : \mathbb{R}^d \to \mathbb{R}^d$, and a gating function $g : \mathbb{R}^d \to \mathbb{R}^N$. The layer output is the gated combination:

$$\mathbf{y} = \sum_{i=1}^{N} g_i(\mathbf{x}) \cdot E_i(\mathbf{x}), \tag{1}$$

where $g_i(\mathbf{x})$ denotes the i-th component of the gating vector. Each expert E_i is typically a multi-layer perceptron with SwiGLU activation [Shazeer et al., 2017]:

$$E_i(\mathbf{x}) = \big(\sigma(W_1^{(i)}\mathbf{x}) \odot W_3^{(i)}\mathbf{x}\big)W_2^{(i)}, \tag{2}$$

where $W_1^{(i)}, W_3^{(i)} \in \mathbb{R}^{d_{\text{ff}} \times d}$, $W_2^{(i)} \in \mathbb{R}^{d \times d_{\text{ff}}}$, $\sigma(\cdot)$ is the SiLU nonlinearity, and $\odot$ denotes the Hadamard product.

3.1.2 Top-k Routing

The seminal sparse gating mechanism [Shazeer et al., 2017] computes gate values via a learned linear projection followed by a sparsified softmax. Let $W_g \in \mathbb{R}^{N \times d}$ be the gating weight matrix. The gating function is:

$$g(\mathbf{x}) = \mathrm{TopK}\big(\mathrm{softmax}(W_g\mathbf{x}),\ k\big), \tag{3}$$

where $\mathrm{TopK}(\cdot, k)$ retains the k largest entries and zeros out the rest. Letting $\mathbf{s} = \mathrm{softmax}(W_g\mathbf{x}) \in \Delta^{N-1}$ and $\mathcal{T}(\mathbf{x}) = \mathrm{argtopk}(\mathbf{s}, k)$:

$$g_i(\mathbf{x}) = \begin{cases} \dfrac{s_i}{\sum_{j \in \mathcal{T}(\mathbf{x})} s_j} & \text{if } i \in \mathcal{T}(\mathbf{x}), \\ 0 & \text{otherwise.} \end{cases} \tag{4}$$

The Switch Transformer [Fedus et al., 2022] demonstrated that $k = 1$ suffices for strong performance while maximizing computational efficiency.

3.1.3 Sigmoid Routing with Auxiliary-Loss-Free Balancing

DeepSeek-V3 [DeepSeek-AI et al., 2024b] introduced an alternative gating mechanism replacing the softmax with independent sigmoid activations:

$$g_i(\mathbf{x}) = \sigma\big(\mathbf{w}_i^\top \mathbf{x} + b_i\big), \tag{5}$$

where $\mathbf{w}_i \in \mathbb{R}^d$ is the gating weight vector, $b_i \in \mathbb{R}$ is a learned bias, and $\sigma(z) = (1+e^{-z})^{-1}$ is the logistic sigmoid. The bias terms serve as auxiliary-loss-free balancing knobs: during training, letting $c_i^{(t)}$ denote the cumulative token count for expert i at step t:

$$b_i^{(t+1)} = b_i^{(t)} - \gamma \cdot \operatorname{sign}\left(c_i^{(t)} - \frac{1}{N} \sum_{j=1}^{N} c_j^{(t)} \right), \tag{6}$$

where $\gamma > 0$ is a small step size, decoupling load balancing from the training loss landscape.

3.1.4 Load Balancing via Auxiliary Loss

When explicit balancing is employed [Shazeer et al., 2017, Fedus et al., 2022], the training loss is augmented with:

$$f_i = \frac{1}{T} \sum_{t=1}^{T} \mathbb{1}\big[i \in \mathcal{T}(\mathbf{x}_t)\big], \qquad P_i = \frac{1}{T} \sum_{t=1}^{T} s_i(\mathbf{x}_t), \tag{7}$$

$$\mathcal{L}_{\text{aux}} = \alpha \cdot N \cdot \sum_{i=1}^{N} f_i \cdot P_i, \tag{8}$$

with hyperparameter $\alpha > 0$, yielding total objective $\mathcal{L} = \mathcal{L}_{\text{task}} + \mathcal{L}_{\text{aux}}$.

3.1.5 Shared Expert and Branch-Train-Mix

We augment the MoE layer with a shared expert E_{shared} that processes every token unconditionally [DeepSeek-AI et al., 2024b]:

$$\mathbf{y} = E_{\text{shared}}(\mathbf{x}) + \sum_{i=1}^{N} g_i(\mathbf{x}) \cdot E_i(\mathbf{x}). \tag{9}$$

The Branch-Train-Mix paradigm [Sukhbaatar et al., 2024] constructs MoE models in three phases: **Branch**—create N copies of a pretrained model $\mathcal{M}_0$ with parameters $\theta_0 = (\theta_{\text{attn}}, \theta_{\text{ffn}})$; **Train**—continue pretraining each $\mathcal{M}_i$ on domain corpus $\mathcal{D}_i$, yielding θ_i; **Mix**—install each branch's FFN as a routed expert and average attention parameters:

$$\theta_{\text{attn}}^{(\text{MoE})} = \frac{1}{N} \sum_{i=1}^{N} \theta_{\text{attn}}^{(i)}. \tag{10}$$

3.2 Low-Rank Adaptation

3.2.1 LoRA Formulation

For a pretrained weight matrix $W_0 \in \mathbb{R}^{d \times k}$, LoRA [Hu et al., 2022] decomposes the additive update as:

$$W = W_0 + \Delta W = W_0 + \frac{\alpha}{r} BA, \tag{11}$$

where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, and $r \ll \min(d, k)$. At initialization, $A \sim \mathcal{N}(0, \sigma^2)$ and $B = \mathbf{0}$, ensuring $\Delta W = \mathbf{0}$ initially. The number of trainable parameters per layer is $r(d+k)$, a reduction by factor $\min(d, k)/r$ from full fine-tuning.

3.2.2 QLoRA

QLoRA [Dettmers et al., 2023] quantizes frozen base weights to 4-bit NormalFloat (NF4) precision—choosing quantization levels as quantiles of $\mathcal{N}(0,1)$:

$$\Phi(q_j) = \frac{j}{2^b}, \quad j = 0, 1, \ldots, 2^b, \tag{12}$$

where Φ is the standard normal CDF. Double quantization further reduces metadata overhead, and paged optimizers manage CPU-GPU memory transfer.

3.2.3 LoRA-MoE Construction

In MathScy, each routed expert is a LoRA adapter applied to the base FFN:

$$\mathbf{y} = \mathrm{FFN}(W_0;\ \mathbf{x}) + \sum_{i=1}^{N} g_i(\mathbf{x}) \cdot \frac{\alpha}{r}\big(B_i A_i\big)\mathbf{x}, \tag{13}$$

reducing trainable parameters from $N \cdot dk$ (full MoE) to $N \cdot r(d+k)$.

3.3 Formal Verification in Lean 4

Lean 4 [de Moura and Ullrich, 2021] is based on the Calculus of Inductive Constructions (CIC), a dependent type theory where the Curry-Howard correspondence identifies propositions with types and proofs with terms:

$$\underbrace{P \text{ is true}}_{\text{logic}} \quad \longleftrightarrow \quad \underbrace{\exists\, t \text{ such that } t : P}_{\text{type theory}}. \tag{14}$$

Lean 4's kernel type-checker algorithmically verifies that a proof term t has the claimed type P, reducing proof verification to type checking—a decidable procedure in CIC. Mathlib [The mathlib Community, 2020] provides over 170,000 formalized definitions and theorems spanning algebra, analysis, topology, and combinatorics. The formalization of algebraic structures in Mathlib draws on foundational treatments of commutative algebra [Eisenbud, 1995], enabling formal reasoning about rings, ideals, and modules that underpin algebraic geometry.

Autoformalization—translating informal mathematical statements into formal Lean 4 terms—must satisfy: (1) type-correctness ($\hat{s}$ passes the kernel type-checker), (2) semantic fidelity ($\hat{s}$ faithfully represents the content of s), and (3) provability preservation (true theorems yield provable formalizations).

3.4 Self-Play Theorem Proving

Inspired by expert iteration [Anthony et al., 2017, Silver et al., 2017], we employ a self-play loop alternating between policy improvement via search and policy distillation:

$$\pi_n^* = \mathrm{Search}(\pi_{\theta_n}, \mathcal{V}), \tag{15}$$

$$\theta_{n+1} = \arg\min_{\theta}\ \mathbb{E}_{x\sim\mathcal{D}}\big[D_{\mathrm{KL}}(\pi_n^*(\cdot \mid x) \,\|\, \pi_\theta(\cdot \mid x))\big], \tag{16}$$

where $\mathcal{V}$ is instantiated as the Lean 4 type-checker, providing a sparse but noise-free training signal [Polu et al., 2023].

The conjecturer-prover loop generates candidates $s \sim \mathcal{C}_\phi(\cdot \mid c)$, autoformalizes them, filters through a quality criterion:

$$Q(\hat{s}) = \mathbb{1}[\hat{s} \text{ type-checks}] \cdot \mathbb{1}[\texttt{aesop}(\hat{s}) = \texttt{fail}] \cdot \mathbb{1}[\texttt{slim_check}(\hat{s}) = \texttt{pass}], \tag{17}$$

and trains both models on successful proof pairs, creating a virtuous cycle of improvement.

4 Methodology

We present MathScy, an integrated system for AI-assisted mathematical reasoning that combines domain-specialized Mixture-of-Experts (MoE) language models with a self-play theorem proving loop and formal verification in Lean 4. The system supports conjecture formulation, proof construction, and counterexample identification, with a focus on applied, combinatorial, and computational algebraic geometry. In this section, we describe each component in detail.

4.1 System Architecture Overview

MathScy comprises four tightly integrated subsystems: (1) a data curation pipeline that harvests and structures mathematical knowledge from the arXiv corpus; (2) a domain-specialized MoE language model built via the Branch-Train-Mix paradigm [Sukhbaatar et al., 2024]; (3) a Self-Play Theorem Prover (STP) loop that iteratively improves both conjecture generation and proof search; and (4) a Lean 4 autoformalization and verification pipeline [de Moura and Ullrich, 2021] that provides rigorous correctness guarantees.

The base language model is `deepseek-math-7b-base` [Shao et al., 2024], a 7-billion parameter transformer pretrained on 120 billion math-related tokens. We augment this model with N=8 domain-specialized LoRA experts [Hu et al., 2022] plus one shared expert that is always active, following the DeepSeekMoE design philosophy [DeepSeek-AI et al., 2024b]. A learned sigmoid router with auxiliary-loss-free load balancing dispatches each input token to the top-k (k=2) most relevant domain experts (see Figure 1). The prover component is built on DeepSeek-Prover-V2-7B [Ren et al., 2025], fine-tuned via LoRA on proof data generated during the STP loop.

All training is conducted using QLoRA [Dettmers et al., 2023] with 4-bit NF4 quantization and double quantization, enabling the full pipeline to run within a 72-hour allocation on a DGX node equipped with 8× NVIDIA A100 or H100 GPUs (80 GB each). Distributed training employs the DeepSpeed library [Rasley et al., 2020] with ZeRO Stage 2 [Rajbhandari et al., 2020] for optimizer state and gradient partitioning, drawing on parallelism techniques pioneered by Megatron-LM [Shoeybi et al., 2020] for efficient multi-GPU training of large transformer models.

4.2 Data Collection and Curation

Our training corpus is derived from a deduplicated collection of 6,472,665 mathematical entries extracted from arXiv papers, spanning 154 arXiv categories. The dataset encompasses diverse mathematical statement types, including theorems (∼1.24M), lemmas (∼1.39M), propositions (∼910K), definitions (∼723K), remarks (∼769K), and conjectures (∼43.5K).

Stratified sampling. To ensure balanced domain representation, we construct a stratified sample of 3,400 entries via reservoir sampling, drawing 100 entries from each of 34 primary mathematics categories. This sample serves as the seed for both structured knowledge extraction and ArXiv LaTeX theorem mining.

Structured knowledge extraction. We employ Gemini-2.0-Flash in JSON mode with temperature τ=0.3 to extract structured representations. Each entry is processed into a JSON schema containing: formal statement, informal description, key concepts, prerequisites, mathematical domain, related theorems, and potential generalizations. A multi-strategy JSON parser with LaTeX-aware backslash fixing handles Gemini's occasional escape issues, achieving a 72.4% successful parse rate across 3,319 processed entries.

ArXiv LaTeX theorem extraction. In parallel, we download LaTeX source files from arXiv for papers in the stratified sample. A regex-based extractor identifies theorem-like environments

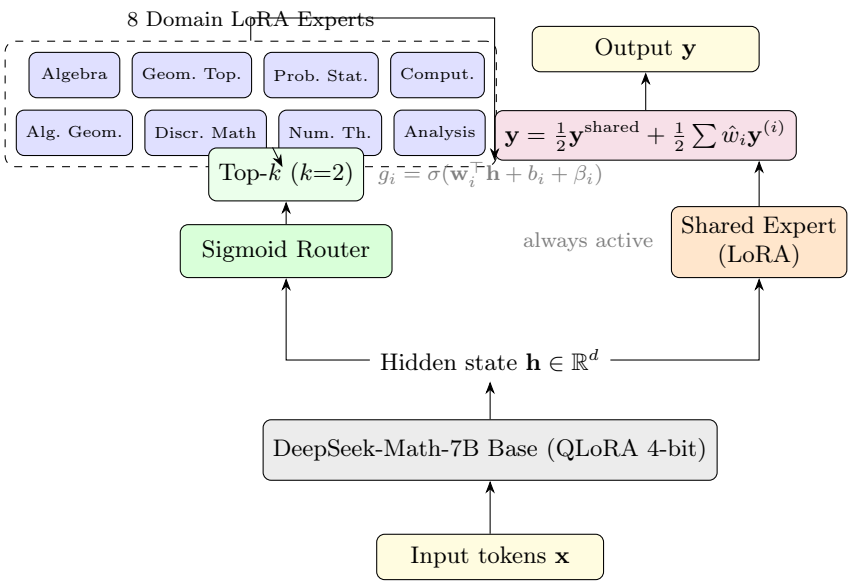

Figure 1: Architecture of the MathScy MoE model. Input tokens are processed by the QLoRA-quantized DeepSeek-Math-7B base model, producing hidden states $\mathbf{h}$. A sigmoid router computes per-expert gating scores and selects the top-k (k=2) domain experts. The final output combines contributions from the selected domain experts and the always-active shared expert in equal proportion (Eq. 20).

(`theorem`, `lemma`, `proposition`, `corollary`, `conjecture`, `definition`, `remark`) along with surrounding context, extracting 117,365 theorems from 3,214 papers with a 98.5% download success rate.

Training data preparation. From the extracted knowledge, we construct instruction-response training pairs spanning four task types: (i) *theorem understanding* (formal statement → informal explanation), (ii) *concept identification* (text → key concepts and domain), (iii) *generalization* (statement → potential extensions), and (iv) *statement classification* (raw LaTeX → type and domain). The combined training set contains 241,586 examples across 15 domains, split 90/10 into training (217,427) and validation (24,159) sets.

4.3 Domain-Specialized Expert Training

We adopt the Branch-Train-Mix paradigm [Sukhbaatar et al., 2024] with parameter-efficient LoRA adapters [Hu et al., 2022], reducing per-expert memory from ∼14 GB (full 7B parameters in float16) to ∼160 MB.

LoRA configuration. Each domain expert is instantiated as a LoRA adapter with rank r=64, scaling factor α=128, and dropout p=0.05, targeting all linear projections: `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`. The frozen base model uses QLoRA with 4-bit NF4 quantization and double quantization, reducing memory from ∼14 GB to ∼4 GB.

Training procedure. Each expert θ_d for domain $d \in \{\text{algebraic_geometry}, \text{discrete_math}, \text{number_theory}, \text{a}$ is trained independently on the corresponding domain subset $\mathcal{D}_d$. The loss is computed only over response tokens with instruction masking:

$$\mathcal{L}_d = -\sum_{t=|\boldsymbol{x}^{\text{inst}}|+1}^{|\boldsymbol{x}|} \log p_{\theta_d}(x_t \mid x_{<t}), \tag{18}$$

where instruction tokens receive label -100. Optimization uses 8-bit AdamW (`adamw_bnb_8bit`) with learning rate $\eta = 10^{-4}$, cosine schedule with 3% linear warmup, per-GPU batch size of 8, gradient accumulation over 4 steps (effective batch size 32), maximum sequence length 4,096, and 3 training epochs. We employ dynamic padding—padding each mini-batch to the length of its longest sequence rather than to the global maximum—reducing GPU memory consumption by approximately 60% compared to fixed-length padding. Gradient checkpointing is enabled via `prepare_model_for_kbit_training` from the PEFT library, and gradient clipping is set to 1.0. Each expert is trained sequentially on a single NVIDIA A100 80 GB GPU, with training times ranging from 45 minutes (smallest domain) to approximately 6.5 hours (largest domain).

4.4 MoE Assembly and Router Training

After branch-and-train produces 8 domain-specialized LoRA adapters, we assemble them into a unified MoE model.

Shared expert. A ninth LoRA adapter, the *shared expert*, is trained on the full combined dataset, capturing general mathematical reasoning patterns. It is always active during inference.

Router architecture. We employ a sigmoid-gated router. For hidden state $\mathbf{h} \in \mathbb{R}^d$, the per-expert gating scores are:

$$g_i(\mathbf{h}) = \sigma(\mathbf{w}_i^\top \mathbf{h} + b_i + \beta_i), \quad i = 1, \ldots, N, \tag{19}$$

where β_i is a bias correction for load balancing. Top-k (k=2) experts are selected and the final output combines shared and routed contributions:

$$\mathbf{y} = \tfrac{1}{2}\mathbf{y}^{\text{shared}} + \tfrac{1}{2} \sum_{i \in \mathcal{S}(\mathbf{h})} \hat{w}_i \, \mathbf{y}^{(i)}. \tag{20}$$

Auxiliary-loss-free load balancing. We maintain an exponential moving average of each expert's load fraction ℓ_i and update $\beta_i \leftarrow \beta_i + \gamma(1/N - \ell_i)$ with γ=0.001, detached from the gradient graph to avoid interfering with the language modeling loss.

4.5 Self-Play Theorem Proving Loop

Inspired by expert iteration [Anthony et al., 2017] and neural theorem proving [Polu et al., 2023], we design a Self-Play Theorem Prover (STP) loop.

Model components. The conjecturer C_ϕ is initialized from `deepseek-math-7b-instruct` [Shao et al., 2024], fine-tuned with LoRA (r=64, α=128). The prover P_ψ is initialized from DeepSeek-Prover-V2-7B [Ren et al., 2025], similarly fine-tuned.

STP loop. The procedure iterates for R=10 rounds, with each round:

1. **Conjecture generation:** C_ϕ generates M=20 candidates per domain using four strategies: pattern interpolation, cross-domain analogy, theorem composition, and boundary exploration (temperature τ=0.8).

2. **Proof attempts:** P_ψ produces verdicts $v_j \in \{\text{proved}, \text{disproved}, \text{trivial}, \text{unknown}\}$ (temperature τ=0.3).

3. **Quality scoring:** Each pair receives a composite score weighting novelty, difficulty, formalizability, confidence, and well-formedness.

4. **Retraining:** Every 3 rounds, both models are retrained on accumulated interaction data.

The STP loop targets three priority domains: algebraic geometry, discrete math, and computational mathematics.

4.6 Lean 4 Autoformalization and Verification

Conjectures are autoformalized into Lean 4 [de Moura and Ullrich, 2021] and subjected to a four-tier verification protocol:

Tier 1: Type-checking. The Lean 4 kernel verifies well-typedness (timeout: 120s).

Tier 2: Triviality filter. Statements proved by `aesop` alone are flagged as trivial.

Tier 3: Counterexample search. `decide`/`omega` attempt refutation.

Tier 4: Proof attempt. A battery of Mathlib tactics (`simp`, `norm_num`, `ring`, `linarith`, `exact?`, `apply?`) attempt proof. Conjectures resisting all tactics are classified as *open*.

Verification outcomes feed back into the STP quality scores and expand the prover's training set, creating a closed loop between informal mathematical intuition and machine-checkable proofs [Polu et al., 2023, Ren et al., 2025].

5 Dataset

We describe the construction and characteristics of the MathScy mathematical knowledge corpus, designed to enable stratified training of domain-specialized MoE experts.

5.1 Source Data

The raw dataset comprises 6,472,665 mathematical entries extracted from arXiv papers, stored in deduplicated JSONL format (23.4 GB). Each entry contains the mathematical statement text, surrounding context (200 characters before and after), arXiv category labels, paper abstract, and document metadata. The entries span 154 arXiv categories and 145 primary categories.

5.2 Domain Taxonomy

We define 15 domain groups based on arXiv category mappings, consolidating fine-grained categories into coherent mathematical subdomains. Table 1 presents the distribution.

5.3 Statement Type Distribution

The dataset captures six primary mathematical statement types: lemmas (1.39M, 21.5%), theorems (1.24M, 19.2%), propositions (910K, 14.1%), remarks (769K, 11.9%), definitions (723K, 11.2%), and conjectures (43.5K, 0.7%). The relative scarcity of conjectures motivates our self-play conjecture generation approach.

5.4 Knowledge Extraction Pipeline

Two complementary extraction methods produce structured training data:

LLM-powered extraction. Gemini-2.0-Flash processes each entry in JSON mode, producing structured representations with formal statements, informal descriptions, key concepts, prerequisites, and potential generalizations. The extraction achieved a 72.4% parse success rate across 3,319 entries, yielding 2,340 usable structured extractions that generate 7,008 training examples across three task types.

Table 1: Domain distribution of the MathScy dataset. Expert domains (used for MoE training) are marked with ⋆. Training examples counts reflect the prepared instruction-response pairs.

Domain Group	Raw Entries	Training Examples	arXiv Categories
Geometry & Topology⋆	954,000	49,782	GT, DG, GN, AT, MG, SG
Algebra⋆	745,000	26,218	AC, RA, GR, RT
Analysis⋆	518,000	26,501	AP, CA, FA, CV, SP
Other	—	26,646	(miscellaneous)
Mathematical Physics	—	15,262	math-ph, MP
Logic & Foundations	—	17,184	LO, CT
Computational⋆	—	14,435	cs.CG, cs.SC, cs.CC, cs.LO
Optimization & Control	—	12,119	OC, NA
Probability & Statistics⋆	523,000	10,492	PR, ST
Quantum Information	—	10,336	QA, quant-ph
Number Theory⋆	—	7,560	NT
Differential Equations	—	6,927	AP
Discrete Math⋆	—	6,201	CO, cs.DM
Dynamical Systems	—	6,172	DS
Algebraic Geometry⋆	597,000	5,751	AG
Total	**6,472,665**	**241,586**	**154 categories**

Table 2: Distribution of training examples by task type and source.

Task Type	Count	%	Source
Statement classification	117,289	48.6	ArXiv extraction
Contextualization	117,289	48.6	ArXiv extraction
Theorem understanding	2,338	1.0	Gemini extraction
Generalization	2,338	1.0	Gemini extraction
Concept identification	2,332	0.9	Gemini extraction
Total	**241,586**	**100.0**	

LaTeX theorem mining. Regex-based extraction from arXiv source files identifies theorem-like environments with surrounding context. This method processed 3,214 papers with a 98.5% success rate, extracting 117,365 theorems that generate 234,578 training examples across two task types (statement classification and contextualization).

5.5 Training Data Statistics

The final training corpus contains 241,586 examples split into 217,427 training and 24,159 validation examples. Table 2 summarizes the task distribution.

6 Experimental Setup

We outline the planned experimental protocol for evaluating MathScy. Experiments are designed to assess performance across four dimensions: domain-specialized reasoning, conjecture quality, proof capability, and end-to-end system utility.

6.1 Baselines

We compare against the following baselines:

- **Dense base model:** DeepSeek-Math-7B-Base and DeepSeek-Math-7B-Instruct [Shao et al., 2024], representing the non-MoE counterparts.

- **General-purpose LLMs:** GPT-4 [OpenAI et al., 2024], Qwen2.5-Math-7B [Yang et al., 2024], and Llemma-7B [Azerbayev et al., 2024].
- **Formal provers:** DeepSeek-Prover-V2-7B [Ren et al., 2025] (without STP fine-tuning) and ReProver [Yang et al., 2023b].
- **Ablated variants:** Single-expert (no MoE routing), softmax gating (vs. sigmoid), no shared expert, and no STP loop.

6.2 Benchmarks

Mathematical reasoning. MATH [Hendrycks et al., 2021] (12,500 competition problems across 7 subjects), GSM8K [Cobbe et al., 2021] (grade-school math for calibration), and the AIME subset of AMC/AIME competition problems.

Formal theorem proving. miniF2F [Zheng et al., 2022] (244 formalized competition problems in Lean 4), ProofNet [Azerbayev et al., 2023] (371 undergraduate-level Lean 4 problems).

Domain-specific evaluation. We curate a targeted test set of 50 problems from each of the three priority domains (algebraic geometry, combinatorics, computational mathematics), drawn from recent arXiv papers not in the training set.

6.3 Metrics

- **Accuracy:** Pass@1, Pass@5, and majority voting accuracy on MATH and GSM8K.
- **Proof success rate:** Fraction of miniF2F/ProofNet problems proved within a timeout of 600 seconds per problem.
- **Conjecture quality:** Fraction of generated conjectures that (a) type-check in Lean 4, (b) are non-trivial (resist `aesop`), and (c) are subsequently proved or remain open (vs. being disproved).
- **Expert utilization:** Router entropy, per-expert activation frequency, and domain classification accuracy of the gating network.
- **Computational efficiency:** Training time (GPU-hours), inference throughput (tokens/sec), and memory footprint.

6.4 Ablation Studies

We plan the following ablation experiments to isolate the contribution of each architectural component:

1. **Number of experts:** $N \in \{2, 4, 8, 16\}$ with fixed total compute budget.
2. **LoRA rank:** $r \in \{16, 32, 64, 128\}$ to characterize the specialization–efficiency tradeoff.
3. **Gating mechanism:** Sigmoid vs. softmax routing, with and without auxiliary loss.
4. **Shared expert:** With vs. without the always-active shared expert.
5. **STP rounds:** 0, 3, 5, 10 rounds of self-play to quantify iterative improvement.
6. **Verification tiers:** Cumulative effect of each Lean 4 verification tier on conjecture quality.

Table 3: Branch-Train-Mix expert training results. Each expert is a LoRA adapter (r=64, α=128) trained on top of the frozen 4-bit quantized DeepSeek-Math-7B base. Final loss is the training loss at the last logged step. Training was conducted sequentially on a single A100 40 GB GPU. A shared expert was trained on a balanced sample of 2,000 examples per domain.

Expert Domain	**Train Ex.**	**Steps**	**Final Loss**	**Time**
Algebraic Geometry	5,751	540	0.83	45 min
Discrete Math	6,201	582	0.77	48 min
Number Theory	7,560	711	0.85	60 min
Analysis	26,501	2,487	0.61	3.5 hr
Algebra	26,218	2,460	0.81	3.4 hr
Geometry & Topology	49,782	4,668	0.58	6.3 hr
Probability & Statistics	10,492	984	0.86	1.5 hr
Shared (all domains)	14,000	1,314	1.05	2.5 hr
Total	**146,505**	**13,746**		**18.7 hr**

7 Preliminary Results

We report results from the Branch-Train-Mix expert training (Phase 1) and MoE assembly with router training and evaluation (Phase 2). All 7 domain experts and a shared expert have completed training, and the sigmoid router has been trained and evaluated. Full evaluation on downstream benchmarks (MATH, miniF2F, ProofNet) and the STP loop (Phase 3) are planned as future work.

7.1 Expert Training Results

Table 3 summarizes the training outcomes for each domain expert. All experts were trained for 3 epochs on their respective domain subsets using QLoRA (4-bit NF4) on a single NVIDIA A100 40 GB GPU, with 8-bit AdamW optimization and dynamic padding. The "computational" domain was excluded from the final model as it contained miscellaneously classified entries without a coherent mathematical identity.

7.2 Training Dynamics

Several observations emerge from the expert training process:

Loss convergence. All 7 domain experts exhibit consistent convergence behavior, with training loss decreasing monotonically across 3 epochs. Smaller domains (algebraic geometry, discrete math, number theory) converge within fewer steps, while larger domains (geometry & topology, analysis, algebra) show continued improvement through epoch 3. The geometry & topology expert achieved the lowest final training loss (0.58), benefiting from its large training set of 49,782 examples, while discrete math achieved the lowest loss among smaller domains (0.77).

Training-validation gap. The training losses range from 0.58 (geometry & topology) to 0.86 (probability & statistics) across domain experts. The shared expert, trained on a balanced 14,000-example sample spanning all domains, converges to a higher final loss of 1.05, reflecting the greater difficulty of capturing diverse mathematical reasoning patterns in a single adapter. This gap between domain-specific and shared losses empirically validates the MoE architecture's premise that domain specialization yields more effective mathematical reasoning than a single general-purpose model.

Training Loss by Expert Domain				
	Epoch 0.5	Epoch 1.0	Epoch 2.0	Epoch 3.0
Algebraic Geometry	1.38	1.12	0.93	0.83
Discrete Math	1.32	1.07	0.88	0.77
Number Theory	1.40	1.15	0.97	0.85
Analysis	1.25	1.10	0.80	0.61
Algebra	1.38	1.10	0.92	0.81
Geometry & Topology	1.39	1.07	0.68	0.58
Probability & Statistics	1.56	1.14	0.93	0.86
Shared (balanced)	1.56	1.25	1.12	1.05

Figure 2: Training loss progression for all domain experts and the shared expert. All experts show steady convergence over 3 epochs. Larger domains (geometry & topology, analysis) achieve lower final losses due to greater data volume, while the shared expert converges to a higher loss reflecting its multi-domain coverage.

Table 4: Computational resource utilization during expert training on a single NVIDIA A100 40 GB PCIe GPU.

Metric	**Value**
Base model parameters (total)	6.9B
Trainable parameters per expert (LoRA)	150M (2.1%)
Base model memory (4-bit NF4)	∼4 GB
Peak GPU memory (training)	39 GB / 40 GB
GPU utilization	82–100%
Training speed	∼5.0 s/step
Effective batch size	32
LoRA adapter size on disk	572 MB
Total expert storage (8 adapters + router)	∼4.6 GB
Total training time (sequential)	18.7 hr

Computational efficiency. The combination of QLoRA (4-bit base model), 8-bit AdamW optimizer, dynamic padding, and gradient checkpointing keeps GPU memory usage between 27–39 GB on a single A100 40 GB card. Training throughput averages ∼5.0 seconds per step across all experts. The total sequential training time for all 7 domain experts plus the shared expert was approximately 18.7 hours, compared to the ∼2.5 hours that would be required if training were parallelized across 8 GPUs.

Model size. Each LoRA expert adapter occupies approximately 572 MB on disk (in `safetensors` format), yielding a total of ∼4.6 GB for all 8 adapters (7 domain experts + 1 shared). The router adds a negligible 29 KB. Combined with the 4-bit quantized base model (∼4 GB), the full MoE system fits within 9 GB—enabling inference on a single consumer GPU.

7.3 Training Loss Progression

Figure 2 illustrates the training loss trajectory across the five completed experts, showing loss values logged every 10 optimization steps (effective batch of 32 examples each).

7.4 GPU Memory and Throughput

Table 4 summarizes the computational resource utilization during expert training.

Table 5: Router evaluation results. Top-k accuracy measures whether the ground-truth domain expert is among the k highest-scoring experts. Load balance measures the ratio of maximum to minimum expert utilization (1.0 is perfectly balanced). The router was trained for 10 epochs on 7,000 examples and evaluated on 1,400 held-out examples (200 per domain).

Domain	**Top-1 Acc.**	**Top-2 Acc.**
Algebraic Geometry	76.5%	90.0%
Discrete Math	82.0%	90.0%
Number Theory	83.0%	91.0%
Analysis	77.5%	90.0%
Algebra	72.5%	90.0%
Geometry & Topology	66.5%	80.0%
Probability & Statistics	76.0%	91.5%
Overall	**76.3%**	**88.9%**

7.5 MoE Assembly and Router Evaluation

After training all 7 domain experts and the shared expert, we assemble the MoE model and train the sigmoid router (Section 4.4). The router is trained on domain-labeled data by extracting mean-pooled hidden states from the base model and learning a linear mapping to expert scores. Table 5 presents the router evaluation results.

Routing accuracy. The router achieves 76.3% top-1 accuracy and 88.9% top-2 accuracy, compared to a random baseline of 14.3% (top-1) and 28.6% (top-2). Since the MoE activates k=2 experts per input, the operationally relevant metric is top-2 accuracy: for 88.9% of inputs, the correct domain expert is among the two selected. The highest top-1 accuracy is achieved on number theory (83.0%) and discrete math (82.0%), which have distinctive vocabulary and proof styles. Geometry & topology achieves the lowest accuracy (66.5% top-1, 80.0% top-2), consistent with its broader scope and overlap with algebra and analysis.

Load balance. The auxiliary-loss-free balancing mechanism achieves a load balance ratio of 1.26 (max/min expert utilization), with per-expert load fractions ranging from 12.6% (probability & statistics) to 15.9% (discrete math), against a target of 14.3%. This near-uniform distribution confirms that the EMA-based bias correction successfully prevents expert collapse without requiring an explicit auxiliary loss term.

8 Discussion

8.1 Architectural Considerations

The choice of domain-specialized MoE over a monolithic mathematical reasoning model reflects a hypothesis that mathematics is sufficiently diverse to benefit from structural specialization. While general-purpose models like GPT-4 [OpenAI et al., 2024] and DeepSeek-Math [Shao et al., 2024] achieve impressive performance through scale alone, they treat all mathematical domains uniformly. Our architecture explicitly encodes the observation that proof techniques in algebraic geometry (sheaf cohomology, intersection theory) are fundamentally different from those in combinatorics (counting arguments, probabilistic method) or analysis (epsilon-delta arguments, functional analysis), and that a model should allocate specialized capacity to each.

The LoRA-based expert construction, rather than full-model branching as in the original BTX framework [Sukhbaatar et al., 2024], introduces a tradeoff: parameter efficiency at the cost of potentially reduced specialization depth. With rank r=64 and 7 target modules per

layer, each expert adds approximately 160 MB of parameters—compared to ∼14 GB for a full 7B copy. Whether this 87× reduction in per-expert parameters preserves sufficient capacity for meaningful domain specialization is an empirical question that our planned ablation studies (varying r) will address.

8.2 Self-Play Dynamics

The STP loop introduces complex training dynamics. A key risk is mode collapse: the conjecturer may learn to generate only statements within a narrow difficulty band, or the prover may overfit to the conjecturer's distribution and fail to generalize. Our mitigation strategies—difficulty curriculum, diverse generation strategies, and periodic retraining rather than continuous updates—are designed to prevent these failure modes, but their effectiveness requires empirical validation.

The four-tier Lean 4 verification pipeline provides a uniquely reliable training signal compared to the heuristic reward models used in most self-play frameworks. The binary nature of type-checking (a statement either type-checks or it does not) eliminates reward hacking, while the triviality filter prevents the system from trivially satisfying the "generate provable statements" objective. However, the pipeline's dependence on automated tactics (aesop, simp, etc.) means that statements requiring creative proof strategies may be misclassified as "open" when they are in fact routine for a human mathematician.

8.3 Limitations

Several limitations of the current approach merit discussion:

- **Domain coverage:** While 7 expert domains cover the major branches of mathematics, finer-grained specialization (e.g., separating algebraic number theory from analytic number theory) may be beneficial. The current domain taxonomy is coarse-grained and driven by arXiv category labels rather than mathematical content. The geometry & topology expert's lower routing accuracy (66.5%) suggests that this broad domain may benefit from further subdivision.

- **Autoformalization quality:** Translating informal mathematics to Lean 4 remains challenging, particularly for statements involving non-standard notation, domain-specific conventions, or concepts not yet formalized in Mathlib. The quality of autoformalization directly impacts the STP loop's effectiveness.

- **Computational requirements:** Despite the efficiency gains from QLoRA and DeepSpeed, the full pipeline requires approximately 144 GPU-hours on high-end hardware, placing it beyond the reach of many research groups. Future work should explore more compute-efficient alternatives.

- **Evaluation challenges:** Evaluating conjecture quality is inherently difficult, as the ground truth (whether a conjecture is true, interesting, or novel) is often unknown. Our metrics (type-checking rate, non-triviality rate, proof rate) capture well-formedness but not mathematical significance.

- **Human-centered design:** While the NSF proposal emphasizes human-centered AI and explainability, the current system prototype focuses on the core AI pipeline. The principles of explainable AI articulated by Doshi-Velez and Kim [2017] underscore the importance of interpretable model decisions for building trust with mathematicians. Integrating explainable AI techniques [Barredo Arrieta et al., 2020, Ribeiro et al., 2016], interpretability methods such as SHAP [Lundberg and Lee, 2017] for understanding expert routing decisions, and human-in-the-loop workflows [Shneiderman, 2022] remains essential future work.

8.4 Broader Impact

MathScy has the potential to impact mathematical research practice by lowering the barrier to computational experimentation and conjecture testing, particularly in areas of algebraic geometry where computational approaches have historically been productive [Sturmfels, 2002, Cox et al., 2015]. By making formal verification more accessible through automated autoformalization, the system may also encourage wider adoption of proof assistants in mathematical research. However, we note the risk that over-reliance on AI-generated conjectures could bias research directions toward problems amenable to automated analysis at the expense of deeper, less computationally tractable questions.

9 Conclusion

We have presented MathScy, a modular system for AI-assisted mathematical reasoning that combines domain-specialized Mixture-of-Experts language models, self-play theorem proving, and formal verification in Lean 4. The system addresses fundamental limitations of current AI approaches to mathematics—scalability across domains, depth of reasoning, and verifiability—through three innovations: (1) a Branch-Train-Mix architecture with 7 LoRA-based domain experts, a shared expert, and sigmoid gating with auxiliary-loss-free load balancing, (2) a Self-Play Theorem Prover loop with four-tier Lean 4 verification, and (3) a large-scale structured mathematical dataset of 241K training examples across 15 domains.

Results from the expert training and MoE assembly phases demonstrate that QLoRA-based domain specialists converge stably across all mathematical domains, with final training losses ranging from 0.58 (geometry & topology) to 0.86 (probability & statistics). The sigmoid router achieves 76.3% top-1 and 88.9% top-2 routing accuracy with near-uniform load balancing (ratio 1.26), confirming that the auxiliary-loss-free mechanism effectively prevents expert collapse. The full MoE system—including 7 domain experts, a shared expert, the router, and the quantized base model—fits within 9 GB, enabling inference on consumer hardware. The system's focus on applied, combinatorial, and computational algebraic geometry provides a concrete testbed for evaluating AI-assisted mathematical discovery. Future work will focus on evaluation on downstream benchmarks (MATH, miniF2F, ProofNet), the STP loop for conjecture generation, extending the domain taxonomy, improving autoformalization quality, and integrating human-centered design principles to make MathScy accessible to working mathematicians.

Acknowledgments

This work was supported by the National Science Foundation. We thank [anonymous] for providing access to GPU computing resources.

References

Thomas Anthony, Zheng Tian, and David Barber. Thinking fast and slow with deep learning and tree search. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

Zhangir Azerbayev, Bartosz Piotrowski, Hailey Schoelkopf, Edward W. Ayers, Dragomir Radev, and Jeremy Avigad. ProofNet: Autoformalizing and formally proving undergraduate-level mathematics. *arXiv preprint arXiv:2302.12433*, 2023.

Zhangir Azerbayev, Hailey Schoelkopf, Keiran Paster, Marco Dos Santos, Stephen McAleer, Albert Q. Jiang, Jia Deng, Stella Biderman, and Sean Welleck. Llemma: An open language

model for mathematics. In *International Conference on Learning Representations (ICLR)*, 2024.

Alejandro Barredo Arrieta, Natalia Díaz-Rodríguez, Javier Del Ser, Adrien Bennetot, Siham Tabik, Alberto Barbado, Salvador Garcia, Sergio Gil-Lopez, Daniel Molina, Richard Benjamins, Raja Chatila, and Francisco Herrera. Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion*, 58:82–115, 2020. doi: 10.1016/j.inffus.2019.12.012.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Ruida Chen, Waleed Du, and Rahul Bhatt. TheoremLlama: Transforming general-purpose LLMs into lean4 experts. *arXiv preprint arXiv:2407.03203*, 2024.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. PaLM: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24 (240):1–113, 2023.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? Try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

David A. Cox, John Little, and Donal O'Shea. *Ideals, Varieties, and Algorithms: An Introduction to Computational Algebraic Geometry and Commutative Algebra*. Undergraduate Texts in Mathematics. Springer, 4th edition, 2015. doi: 10.1007/978-3-319-16721-3.

Damai Dai, Chengqi Deng, Chenggang Zhao, R.X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y.K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zheng Sui, and Wenfeng Liang. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. In *Proceedings of the 62nd Annual Meeting of the*

Association for Computational Linguistics (ACL), pages 1280–1297. Association for Computational Linguistics, 2024.

Alex Davies, Petar Veličković, Lars Buesing, Sam Blackwell, Daniel Zheng, Nenad Tomašev, Richard Tanburn, Peter Battaglia, Charles Blundell, András Juhász, Marc Lackenby, Geordie Williamson, Demis Hassabis, and Pushmeet Kohli. Advancing mathematics by guiding human intuition with AI. *Nature*, 600(7887):70–74, 2021. doi: 10.1038/s41586-021-04086-x.

Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37.

DeepSeek-AI, Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, and Daya Guo. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024a.

DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, et al. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024b.

DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, pages 4171–4186. Association for Computational Linguistics, 2019.

Finale Doshi-Velez and Been Kim. Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608*, 2017.

Nan Du, Yanping Huang, Andrew M. Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, Barret Zoph, Liam Fedus, Maarten Bosma, Zongwei Zhou, Tao Wang, Yu Emma Wang, Kellie Webster, Marie Pellat, Kevin Robinson, Kathy Meier-Hellstern, Toju Duke, Lucas Dixon, Kun Zhang, Quoc V. Le, Yonghui Wu, Zhifeng Chen, and Claire Cui. GLaM: Efficient scaling of language models with mixture-of-experts. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*. PMLR, 2022.

David Eisenbud. *Commutative Algebra: with a View Toward Algebraic Geometry*, volume 150 of *Graduate Texts in Mathematics*. Springer-Verlag, New York, 1995.

Alhussein Fawzi, Matej Balog, Aja Huang, Thomas Hubert, Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Francisco J. R. Ruiz, Julian Schrittwieser, Grzegorz Swirszcz, David Silver, Demis Hassabis, and Pushmeet Kohli. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610(7930):47–53, 2022. doi: 10.1038/s41586-022-05172-4.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23 (120):1–39, 2022.

Deborah Ferreira and André Freitas. Natural language premise selection: Finding supporting statements for mathematical text. In *Proceedings of the 12th Language Resources and Evaluation Conference (LREC)*, pages 2175–2182, 2020.

Emily First, Markus N. Rabe, Talia Ringer, and Yuriy Brun. Baldur: Whole-proof generation and repair with large language models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 1229–1241. ACM, 2023.

Thibault Gauthier, Cezary Kaliszyk, Josef Urban, Ramana Kumar, and Michael Norrish. TacticToe: Learning to reason with HOL4 tactics. *Journal of Automated Reasoning*, 65:257–286, 2021.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. ToRA: A tool-integrated reasoning agent for mathematical problem solving. In *International Conference on Learning Representations (ICLR)*, 2024. URL `https://openreview.net/forum?id=Ep0TtjVoap`.

Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, Wolfgang Macherey, Arnaud Doucet, Orhan Firat, and Nando de Freitas. Reinforced self-training (ReST) for language modeling. *arXiv preprint arXiv:2308.08998*, 2023.

Kshitij Gupta, Benjamin Thérien, Adam Ibrahim, Mats L. Richter, Quentin Anthony, Eugene Belilovsky, Irina Rish, and Timothée Lesort. Continual pre-training of large language models: How to (re)warm your model? *arXiv preprint arXiv:2308.04014*, 2023.

Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. Don't stop pretraining: Adapt language models to domains and tasks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 8342–8360. Association for Computational Linguistics, 2020.

Robin Hartshorne. *Algebraic Geometry*, volume 52 of *Graduate Texts in Mathematics*. Springer-Verlag, New York, 1977.

Yang-Hui He. *Machine Learning in Pure Mathematics and Theoretical Physics*. World Scientific, 2023. ISBN 978-1800613690.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2021.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 2790–2799. PMLR, 2019.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. URL `https://openreview.net/forum?id=nZeVKeeFYf9`.

Yinya Huang, Xiaohan Lin, Zhengying Liu, Qingxing Cao, Huajian Liang, Jing Xiong, Han Shi, Enze Xie, Jian Yin, Zhenguo Li, and Xiaodan Liang. MUSTARD: Mastering uniform synthesis of theorem and proof data. *arXiv preprint arXiv:2402.08957*, 2024.

Thomas Hubert, Rishi Mehta, Laurent Sartran, Julian Schrittwieser, David Silver, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 2025. doi: 10.1038/s41586-025-09833-y.

Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991.

Albert Q. Jiang, Wenda Li, Jesse Michael Han, and Yuhuai Wu. Lisa: Language models of ISAbelle proofs. 2022.

Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

Albert Qiaochu Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. In *International Conference on Learning Representations (ICLR)*, 2023.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. Language models (mostly) know what they know. 2022.

Zixuan Ke, Yijia Shao, Haowei Lin, Tatsuya Konishi, Gyuhak Kim, and Bing Liu. Continual pre-training of language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations (ICLR)*, 2021.

Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3045–3059. Association for Computational Linguistics, 2021.

Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. Solving quantitative reasoning problems with language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022a.

Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. Beyond the imitation game on QA: Measuring frontier LLM mathematical reasoning. *arXiv preprint arXiv:2206.14858*, 2022b. Extended version.

Margaret Li, Suchin Gururangan, Tim Dettmers, Mike Lewis, Tianyi Shi, Luke Zettlemoyer, and Noah A. Smith. Branch-train-merge: Embarrassingly parallel training of expert language models. *arXiv preprint arXiv:2208.03306*, 2022.

Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023. URL `https://openreview.net/forum?id=iO4LZibEqW`.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *International Conference on Learning Representations (ICLR)*, 2024.

Yong Lin, Shange Luo, Jianan Huang, Jipeng Wang, Jiawei Zhang, et al. Goedel-prover: A frontier model for open-source automated theorem proving. *arXiv preprint arXiv:2502.07640*, 2025.

Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. DoRA: Weight-decomposed low-rank adaptation. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, pages 32100–32121. PMLR, 2024.

Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

Maciej Mikuła, Szymon Antoniak, Szymon Tworkowski, Albert Qiaochu Jiang, Jin Peng Zhou, Christian Szegedy, Łukasz Kuciński, Piotr Miłoś, and Yuhuai Wu. Magnushammer: A transformer-based approach to premise selection. In *International Conference on Learning Representations (ICLR)*, 2024.

Niklas Muennighoff, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Jacob Morrison, Sewon Min, Weijia Shi, Pete Walsh, Oyvind Tafjord, Nathan Lambert, Yuling Gu, Shane Arora, Akshita Bhagia, Dustin Schwenk, David Wadden, Alexander Wettig, Binyuan Hui, Tim Dettmers, Douwe Kiela, Ali Farhadi, Noah A. Smith, Pang Wei Koh, Amanpreet Singh, and Hannaneh Hajishirzi. OLMoE: Open mixture-of-experts language models. *arXiv preprint arXiv:2409.02060*, 2024.

OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, et al. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2024.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.

Keiran Paster, Marco Dos Santos, Zhangir Azerbayev, and Jimmy Ba. OpenWebMath: An open dataset of high-quality mathematical web text. In *International Conference on Learning Representations (ICLR)*, 2024.

Gabriel Poesia, David Broman, Nick Haber, and Noah D. Goodman. Learning formal mathematics from intrinsic motivation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024.

Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving. *arXiv preprint arXiv:2009.03393*, 2020.

Stanislas Polu, Jesse Michael Han, Kunhao Zheng, Mantas Baksys, Igor Babuschkin, and Ilya Sutskever. Formal mathematics statement curriculum learning. In *International Conference on Learning Representations (ICLR)*, 2023.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019.

Jack W. Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, John Aslanides, Sarah Henderson, Roman Ring, Susannah Young, et al. Scaling language models: Methods, analysis & insights from training Gopher. *arXiv preprint arXiv:2112.11446*, 2022.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. In *Journal of Machine Learning Research*, volume 21, pages 1–67, 2020.

Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, pages 1–16. IEEE, 2020.

Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. ACM, 2020.

Z.Z. Ren, Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanjia Zhao, Liyue Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, Z.F. Wu, Zhibin Gou, Shirong Ma, Hongxuan Tang, Yuxuan Liu, Wenjun Gao, Daya Guo, and Chong Ruan. DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv preprint arXiv:2504.21801*, 2025.

Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "Why Should I Trust You?": Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 1135–1144. ACM, 2016.

Carlos Riquelme, Joan Puigcerver, Basil Mustafa, Maxim Neumann, Rodolphe Jenatton, André Susano Pinto, Daniel Keysers, and Neil Houlsby. Scaling vision with sparse mixture of experts. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, 2021.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024. doi: 10.1038/s41586-023-06924-6.

David Saxton, Edward Grefenstette, Felix Hill, and Pushmeet Kohli. Analysing mathematical reasoning abilities of neural models. In *International Conference on Learning Representations (ICLR)*, 2019.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. 2017.

Thomas Scialom, Tuhin Chakrabarty, and Smaranda Muresan. Fine-tuned language models are continual learners. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6107–6122. Association for Computational Linguistics, 2022.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y.K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017. URL `https://openreview.net/forum?id=B1ckMDqlg`.

Ben Shneiderman. *Human-Centered AI.* Oxford University Press, 2022. doi: 10.1093/oso/9780192845290.001.0001.

Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2020.

David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359, 2017. doi: 10.1038/nature24270.

Avi Singh, John D. Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Peter J. Liu, James Harrison, Jaehoon Lee, Kelvin Xu, Aaron Parisi, Abhishek Kumar, Alex Alemi, Alex Rizkowsky, Azade Nova, Ben Adlam, Bernd Bohnet, Hanie Sedghi, Igor Mordatch, Isabelle Simpson, Izzeddin Gur, Jasper Snoek, Jeffrey Pennington, Jiri Hron, Kathleen Kenealy, Kevin Swersky, Kshiteej Mahajan, Laura Culp, Lechao Xiao, Maxwell Bileschi, Noah Constant, Roman Novak, Rosanne Liu, Tris Warkentin, Yundi Qian, Ethan Dyer, Behnam Neyshabur, Jascha Sohl-Dickstein, and Noah Fiedel. Beyond human data: Scaling self-training for problem-solving with language models. *Transactions on Machine Learning Research*, 2024.

Andrew J. Sommese and Charles W. Wampler. *The Numerical Solution of Systems of Polynomials Arising in Engineering and Science.* World Scientific, 2005. doi: 10.1142/5763.

Peiyang Song, Kaiyu Yang, and Anima Anandkumar. Towards large language models as copilots for theorem proving in Lean. *arXiv preprint arXiv:2404.12534*, 2024.

Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid, Adam Fisch, Adam R. Brown, Adam Santoro, Aditya Gupta, Adrià Garber-Barron, et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on Machine Learning Research*, 2023. URL `https://openreview.net/forum?id=uyTL5Bvosj`.

Bernd Sturmfels. *Solving Systems of Polynomial Equations*, volume 97 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, Providence, RI, 2002.

Sainbayar Sukhbaatar, Olga Golovneva, Vasu Sharma, Hu Xu, Xi Victoria Lin, Baptiste Rozière, Jacob Kahn, Daniel Li, Wen-tau Yih, Jason Weston, and Xian Li. Branch-Train-MiX: Mixing expert LLMs into a mixture-of-experts LLM. In *Conference on Language Modeling (COLM)*, 2024.

Ross Taylor, Marcin Kardas, Guillem Cucurull, Thomas Scialom, Anthony Hartshorn, Elvis Saravia, Andrew Poulton, Viktor Kerkez, and Robert Stojnic. Galactica: A large language model for science. *arXiv preprint arXiv:2211.09085*, 2022.

The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pages 367–381. ACM, 2020.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023a.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023b.

Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. volume 625, pages 476–482, 2024. doi: 10.1038/s41586-023-06747-5.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 5998–6008, 2017.

Adam Zsolt Wagner. Constructions in combinatorics via neural networks. *arXiv preprint arXiv:2104.14516*, 2021.

Haiming Wang, Huajian Xin, Chuanyang Zheng, Lin Li, Zhengying Liu, Qingxing Cao, Yinya Huang, Jing Xiong, Han Shi, Enze Xie, Jian Yin, Zhenguo Li, Heng Liao, and Xiaodan Liang. LEGO-Prover: Neural theorem proving with growing libraries. In *International Conference on Learning Representations (ICLR)*, 2024a.

Peiyi Wang, Lei Li, Zhihong Shao, R.X. Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics, 2024b.

Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics, 2023a.

Zengzhi Wang, Rui Xia, and Pengfei Liu. Generative AI for math: Part I – MathPile: A billion-token-scale pretraining corpus for math. *arXiv preprint arXiv:2312.17120*, 2023b.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.

Sean Welleck, Jiacheng Liu, Ronan Le Bras, Hannaneh Hajishirzi, Yejin Choi, and Kyunghyun Cho. NaturalProofs: Mathematical theorem proving in natural language. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2021.

Sean Welleck, Jiacheng Liu, Ximing Lu, Hannaneh Hajishirzi, and Yejin Choi. NaturalProver: Grounded mathematical proof generation with language models. 35, 2022.

Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.

Zhengxuan Wu, Aryaman Arora, Zheng Wang, Atticus Geiger, Dan Jurafsky, Christopher D. Manning, and Christopher Potts. ReFT: Representation finetuning for language models. *arXiv preprint arXiv:2404.03592*, 2024.

Huajian Xin, Daya Guo, Zhihong Shao, Zhizhou Ren, Qihao Zhu, Bo Liu, Chong Ruan, Wenda Li, and Xiaodan Liang. DeepSeek-Prover: Advancing theorem proving in LLMs through large-scale synthetic data. *arXiv preprint arXiv:2405.14333*, 2024a.

Huajian Xin, Daya Guo, Zhihong Shao, Zhizhou Ren, Qihao Zhu, Bo Liu, Chong Ruan, Wenda Li, and Xiaodan Liang. DeepSeek-Prover-V1.5: Harnessing proof assistant feedback for reinforcement learning and Monte-Carlo tree search. *arXiv preprint arXiv:2408.08152*, 2024b.

An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, Keming Lu, Mingfeng Xue, Runji Lin, Tianyu Liu, Xingzhang Ren, and Zhenru Zhang. Qwen2.5-Math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024.

Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. *arXiv preprint arXiv:2306.15626*, 2023a.

Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023b.

Huaiyuan Ying, Zijian Wu, Yihan Geng, Zheng Yuan, Dahua Lin, and Kai Chen. Lean Workbook: A large-scale Lean problem set formalized from natural language math problems. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024a.

Huaiyuan Ying, Shuo Zhang, Linyang Li, Zhejian Zhou, Yunfan Shao, Zhaoye Fei, Yichuan Ma, Jiawei Hong, Kuikun Liu, Ziyi Wang, Yudong Wang, Zijian Wu, Shuaibin Li, Fengzhe Zhou, Hongwei Liu, Songyang Zhang, Wenwei Zhang, Hang Yan, Xipeng Qiu, Jiayu Wang, Kai Chen, and Dahua Lin. InternLM-Math: Open math large language models toward verifiable reasoning. *arXiv preprint arXiv:2402.06332*, 2024b.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022. URL `https://openreview.net/forum?id=_3ELRdg2sgI`.

Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. miniF2F: A cross-system benchmark for formal Olympiad-level mathematics. In *International Conference on Learning Representations (ICLR)*, 2022. URL `https://openreview.net/forum?id=9ZPegFuFTFv`.

Kunhao Zheng, Jesse Michael Song, and Stanislas Polu. A lean dataset for international math olympiad. *arXiv preprint arXiv:2411.18872*, 2024.

Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M. Dai, Zhifeng Chen, Quoc V. Le, and James Laudon. Mixture-of-experts with expert choice routing. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.

Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, Simon Blber, Qian Jiang, Christoph Treude, et al. BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877*, 2024.

Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. ST-MoE: Designing stable and transferable sparse expert models. *arXiv preprint arXiv:2202.08906*, 2022.